

MINI PROJECT REPORT

ON

Handwritten Digit Recognition Using Convolutional Neural Network (CNN)

Submitted by

R.DHARINI

NC.SC.U4CSE24151

Submitted to

Mrs.Hari Priya K H

For

23CSE473 – Neural Networks and Deep Learning

V Semester

B.Tech. CSE

School of Computing

Amrita Vishwa Vidyapeetham, Nagercoil

INDEX

S.No	Contents	Page No.
1	Problem Statement	3
2	Objective	3
3	Existing System	4
4	Novelty / Proposed System	4,5
5	Code Implementation	5-7
6	Output / Results	7-9
7	Conclusion	9
8	GitHub Repository Link	9

1. PROBLEM STATEMENT:

Handwritten digit recognition is an important application of artificial intelligence and image processing. It involves identifying numerical digits written by different individuals in various handwriting styles. Manual recognition of handwritten digits can be time-consuming and may lead to errors, especially when dealing with large volumes of data. Traditional machine learning approaches require manual feature extraction, which reduces efficiency and accuracy. Therefore, there is a need for an automated system that can accurately recognize handwritten digits using deep learning techniques. This project aims to develop a Convolutional Neural Network (CNN) based model that can automatically identify handwritten digits from images and classify them into one of the ten digit classes (0–9).

2. OBJECTIVE:

The main objectives of this project are:

- To develop a Handwritten Digit Recognition system using Convolutional Neural Network (CNN).
 - To train the model using the MNIST handwritten digit dataset.
 - To recognize and classify handwritten digits from 0 to 9 accurately.
 - To automatically extract image features using deep learning techniques.
 - To improve the accuracy of digit recognition compared to traditional methods.
 - To evaluate the performance of the CNN model using testing data.
 - To analyze the effectiveness of deep learning in image classification tasks.
 - To achieve high prediction accuracy for handwritten digit recognition.
-

3.EXISTING SYSTEM:

Traditional handwritten digit recognition systems mainly rely on machine learning algorithms that require manual feature extraction. In such systems, image features such as edges, shapes, and patterns must be identified before classification. These methods often struggle when handwriting styles vary significantly between individuals. The accuracy of traditional systems is generally lower when compared to deep learning models. In addition, manual feature engineering increases the complexity of the system and requires domain expertise. As the size of the dataset increases, the performance and scalability of traditional approaches become limited.

Limitations of Existing System

- Requires manual feature extraction.
- Lower accuracy for complex handwriting patterns.
- Difficult to handle large datasets efficiently.
- Performance depends heavily on selected features.
- Less effective compared to deep learning approaches.

4. NOVELTY / PROPOSED SYSTEM:

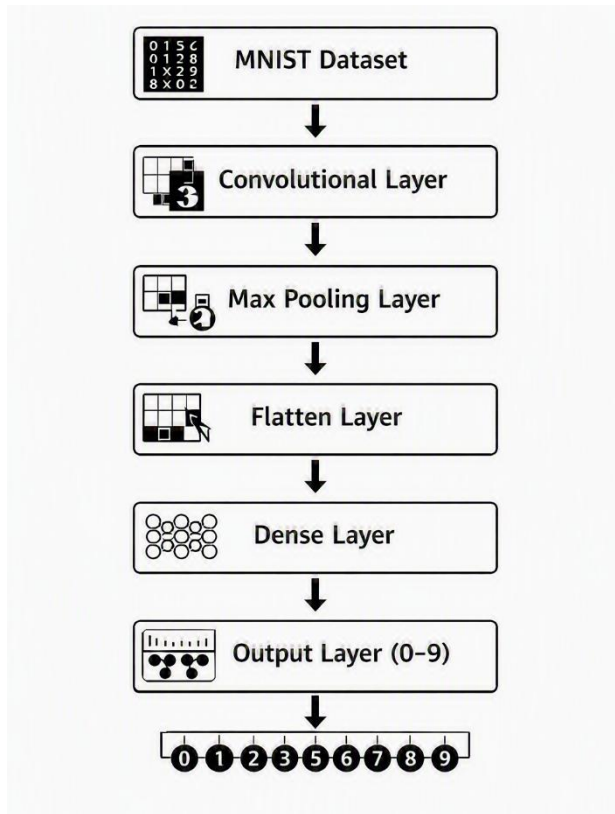
The proposed system utilizes a Convolutional Neural Network (CNN) to recognize handwritten digits automatically. CNN is a specialized deep learning architecture designed for image processing and pattern recognition tasks. Unlike traditional methods, CNN automatically extracts relevant features from images without requiring manual intervention. The MNIST dataset is used for training and testing the model. The proposed system consists of convolution layers, pooling layers, flatten layers, and dense layers that work together to identify patterns within handwritten digit images. By leveraging deep learning techniques, the system achieves high classification accuracy and provides efficient recognition of handwritten digits.

Advantages of Proposed System

- Automatic feature extraction.
- High classification accuracy.

- Reduced human intervention.
- Fast and efficient prediction.
- Suitable for large-scale image datasets.
- Robust performance across different handwriting styles.

System Architecture:



5.CODE IMPLEMENTATION:

The proposed system was implemented using Python and TensorFlow/Keras in Google Colab. The MNIST dataset was loaded and pre-processed by normalizing the images and reshaping them for CNN input. A Convolutional Neural Network (CNN) was created using convolution, max pooling, flatten, and dense layers. The model was trained for 5 epochs and evaluated using the test dataset. The system performance was analysed using accuracy, digit prediction, confusion matrix, and accuracy graphs.

Technologies Used – Python, TensorFlow, Keras, NumPy, Matplotlib & Google Colab

5.1 Dataset Loading and Preprocessing:

The MNIST dataset was loaded using TensorFlow. The images were reshaped and normalized. One-hot encoding was applied to the output labels for multi-class classification.

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.utils import to_categorical

# Load MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()
print("Training Samples:", len(x_train))
print("Testing Samples:", len(x_test))

# Reshape
x_train = x_train.reshape(-1, 28, 28, 1) / 255.0
x_test = x_test.reshape(-1, 28, 28, 1) / 255.0

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# CNN Model
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

# Compile
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Train
model.fit(
    x_train,
    y_train,
    epochs=5,
    validation_data=(x_test, y_test)
)

# Evaluate
loss, accuracy = model.evaluate(x_test, y_test)
print("Test Accuracy =", accuracy)
```

5.2 Displaying Sample Handwritten Digit:

A sample handwritten digit from the MNIST dataset was displayed to visualize the input image used for training and testing.

```
import matplotlib.pyplot as plt

plt.imshow(x_train[0].reshape(28, 28), cmap='gray')
plt.title("Handwritten Digit")
plt.axis('off')
plt.show()
```

5.3 Digit Prediction:

The trained CNN model was used to predict the digit present in the test image. The predicted digit was compared with the actual digit.

```

import matplotlib.pyplot as plt
import numpy as np

plt.imshow(x_test[0].reshape(28,28), cmap='gray')
plt.axis('off')
plt.show()

prediction = model.predict(x_test[0].reshape(1,28,28,1), verbose=0)

print("Predicted Digit:", np.argmax(prediction))
print("Actual Digit:", np.argmax(y_test[0]))

```

5.4 Confusion Matrix Generation:

A confusion matrix was generated to evaluate the classification performance of the CNN model and identify correctly and incorrectly classified digits.

```

from sklearn.metrics import confusion_matrix
import numpy as np

y_pred = model.predict(x_test, verbose=0)
y_pred = np.argmax(y_pred, axis=1)
y_true = np.argmax(y_test, axis=1)

cm = confusion_matrix(y_true, y_pred)
print(cm)

```

5.5 Accuracy Graph Generation:

Training accuracy and validation accuracy graphs were plotted to analyze the learning performance of the model across different epochs.

```

import matplotlib.pyplot as plt

plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])

plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(['Training Accuracy', 'Validation Accuracy'])

plt.show()

```

6. OUTPUT / RESULTS:

Dataset loading output:

```

... Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 1s 0us/step
Training Samples: 60000
Testing Samples: 10000
/usr/local/lib/python3.13/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape` argument to `Layer.__init__` (activity_regularizer=activity_regularizer, **kwargs)

```

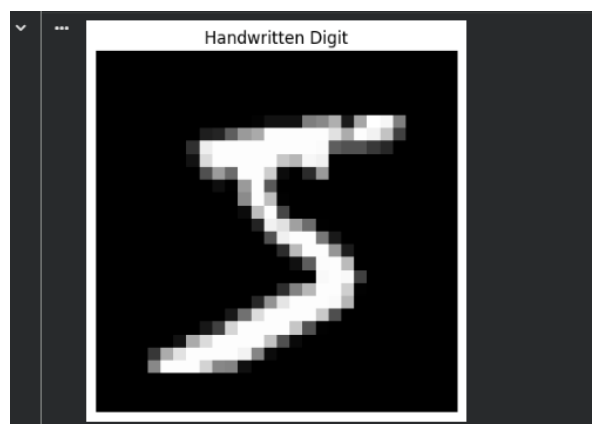
Training output:

```
Epoch 1/5
1875/1875 ————— 24s 12ms/step - accuracy: 0.9547 - loss: 0.1533 - val_accuracy: 0.9786 - val_loss: 0.0698
Epoch 2/5
1875/1875 ————— 23s 13ms/step - accuracy: 0.9841 - loss: 0.0515 - val_accuracy: 0.9821 - val_loss: 0.0544
Epoch 3/5
1875/1875 ————— 23s 12ms/step - accuracy: 0.9894 - loss: 0.0330 - val_accuracy: 0.9856 - val_loss: 0.0414
Epoch 4/5
1875/1875 ————— 23s 12ms/step - accuracy: 0.9931 - loss: 0.0221 - val_accuracy: 0.9826 - val_loss: 0.0551
Epoch 5/5
1875/1875 ————— 41s 12ms/step - accuracy: 0.9956 - loss: 0.0143 - val_accuracy: 0.9845 - val_loss: 0.0573
313/313 ————— 1s 4ms/step - accuracy: 0.9845 - loss: 0.0573
Test Accuracy = 0.984499990940094
```

Final accuracy output:

```
Test Accuracy = 0.984499990940094
```

MNIST digit image:



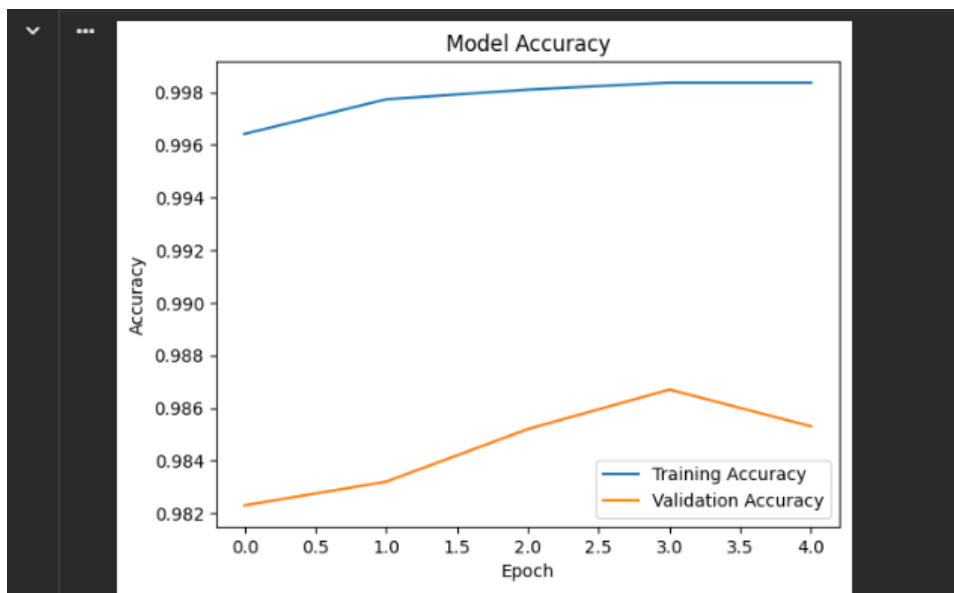
Prediction result:



Confusion Matrix:

```
[[ 974   1   0   0   0   0   3   1   1   0]
 [   0 1132   0   0   0   1   1   1   0   0]
 [   4   5 1007   1   2   0   2   5   5   1]
 [   0   0   0  987   0  18   0   2   3   0]
 [   0   2   0   0  973   0   1   1   0   5]
 [   1   0   0   1   0  888   2   0   0   0]
 [   1   3   0   0   1   5  946   0   2   0]
 [   1   5   8   0   0   1   0 1008   1   4]
 [   2   1   0   0   0   6   1   1  960   3]
 [   3   6   0   2   6  12   0   4   6  970]]
```

Graph:



7. CONCLUSION:

In this project, a Handwritten Digit Recognition System was successfully developed using a Convolutional Neural Network. The MNIST dataset was used to train and evaluate the model. The CNN architecture automatically extracted important image features and accurately classified handwritten digits from 0 to 9. Experimental results showed that the model achieved a high testing accuracy of 98.45%, demonstrating the effectiveness of deep learning techniques for image classification problems. The project highlights the importance of CNNs in modern artificial intelligence applications and their ability to provide reliable and efficient recognition systems. Future enhancements may include recognizing handwritten alphabets, words, and real-time handwritten inputs collected from users.

8. GITHUB REPOSITORY LINK:

<https://github.com/DHARINIRAJSEKAR/Handwritten-Digit-Recognition-CNN.git>
